

ZHAW – Zurich University of Applied Sciences

School of Engineering

Drone Control Tuning

Development of an Offline Betaflight Drone Control Tuner

Autor:	Yuri Bianchi, Dario Jurietti and Janick Dort
Modul / PA:	Project Thesis
Supervisor:	Michael Ernst Peter
Co-Supervisor:	Dr. Prof. Ruprecht Altenburger

Winterthur, 2. Dezember 2025

1 Abstract

This project focuses on the development of a method for offline tuning of gyro controllers for FPV racing drones based on the open-source firmware Betaflight. The quality of the gyro control loop has a direct influence on flight stability, the suppression of disturbances such as prop wash, and the agility of the drone. Until now, rate controller tuning has mostly been performed through time-consuming trial and error.

The aim of the project is to develop a modular, open-source solution for offline tuning based on an existing proof of concept. The code will be implemented in MATLAB, accompanied by a lightweight Python version, and supplemented with detailed Markdown documentation and examples on GitHub. The project aims to provide the community with a simple way to efficiently optimise drone parameters and thereby improve flight performance.

The results will be published in a public GitHub repository and documented in a report. In addition to the software development, the project also includes practical work with FPV drones to validate the developed methods.

2 Foreword

This document summarizes the results of our project thesis, which we worked on during the fall semester 2025 at the ZHAW School of Engineering. For us as systems engineering students, this project provided an ideal opportunity to apply advanced control theory and signal processing methods to a real, highly dynamic system. FPV drones present an interesting engineering challenge due to their highly dynamic behaviour and the need for precise control. This motivated us to investigate mathematical modelling, control engineering aspects, and offline tuning methods aimed at improving flight performance.

Our motivation was to translate the theoretical control concepts we learned over the past three years to a real, highly dynamic system. We also aimed to develop a tool that not only meets engineering standards but also provides real value to the FPV community by replacing subjective tuning with objective data analysis.

This work was carried out at the Institute of Mechatronic Systems (IMS) at the ZHAW School of Engineering. We would like to express our gratitude to our supervisor, Michael Peter, for his guidance and support throughout this project. His expertise, constructive feedback, and willingness to take the time to assist us in developing complex algorithms were invaluable.

Inhaltsverzeichnis

1	Abstract	2
2	Foreword	2
3	Introduction	5
3.1	Initial situation	5
3.2	Task and Objective	5
4	Preparation	6
4.1	Chirp	6
4.1.1	Chirp Concept	6
4.1.2	Chirp on a drone	6
4.1.3	Chirp programming	7
5	Theory	8
5.1	Signal Processing	8
5.1.1	Data Import	8
5.1.2	Handle High-Resolution Data	8
5.1.3	Time Conversion and Sampling Rate Verification	8
5.1.4	Application of Rotational Filtering (Apply Rotfiltfilt)	8
5.1.5	Estimation of Frequency Response	8
5.2	Control System	9
5.2.1	Transfer Function Estimation	9
5.2.2	Closed Loop Calculation	9
5.2.3	Filters and Controllers	9
5.2.4	Compensation of the I-Term	9
5.3	Evaluation	10
5.3.1	Step Response	10
5.3.2	Spectra Analysis	10
5.3.3	Spectrum Analysis	10
6	Challenges and Methodological Approach	11
6.1	Simulations	11
6.1.1	Simulation of Betaflight Controller Order	11
6.1.2	Simulation PID Controller	11
6.1.3	Simulation Frequency Response	11
6.1.4	Simulation Spectra	11
6.1.5	Simulation Spectrogram	11
6.2	Building an FPV drone	12
6.3	Tuning an FPV drone	12
6.4	Decode	13
6.5	Code Structure and programming style	14
6.5.1	OOP vs. Functional Approach	14
6.5.2	Final Structure Concept	14
6.6	Conversion MATLAB to Python	15
6.7	Python in MATLAB	15
6.8	Transfer MATLAB Library to Python	15
7	Results	16

8	Discussion and next steps	16
8.1	Discussion	16
8.2	Next Steps	16
8.3	Conclusion	16
9	Directories	17

3 Introduction

Text...

3.1 Initial situation

Text...

3.2 Task and Objective

Text...

4 Preparation

4.1 Chirp

4.1.1 Chirp Concept

A chirp is an input signal that sweeps continuously over a defined range (e.g. from a low frequency to a high frequency). This signal excites the system on multiple frequencies in one run and thus lets you observe the frequency response and step response, the foundation of controller tuning. Mathematically, a chirp signal can be represented as a sinusoidal wave with instantaneous frequency that varies linearly or exponential with time. For more information about the mathematical background, please have a look in the document [Chirp Signal](#).

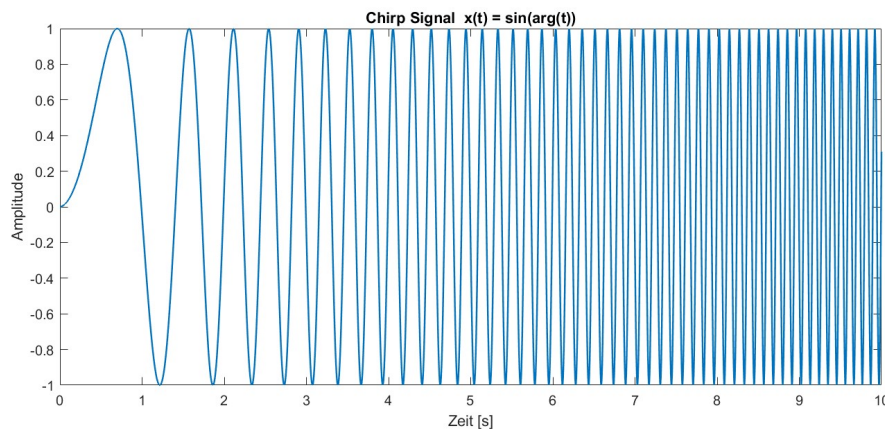


Abbildung 1: Chirp Signal example

4.1.2 Chirp on a drone

In the context of tuning a drone's flight controller, an exponential sweep is used. This is due to the dominant low-frequency dynamics, where low frequencies exhibit more predictable responses, allowing the sweep to allocate more time to higher frequencies for better resolution of system behavior. Additionally, multirotor vibration resonances are typically spaced logarithmically, making exponential sweeps more suited for efficient excitation across the frequency range.

The process involves these steps:

1. **Signal Injection:** The chirp signal is added to the pilot's stick input. This composite signal becomes the setpoint for the PID controller.
2. **System Response:** The drone attempts to follow this rapidly changing setpoint. The motors react, and the drone will oscillate at varying frequencies corresponding to the chirp signal.
3. **Data Logging:** While the chirp is active, high-resolution data from the gyroscope and the motor outputs are recorded using Betaflight's Blackbox feature. This data captures how the drone responds to the control inputs at different frequencies.

The recorded log file, containing the input chirp and the corresponding gyroscope output, is then used for offline analysis to determine the frequency response of the system.

4.1.3 Chirp programming

The implementation of the chirp signal in Betaflight is straightforward. You call the function `chirpInit()` and pass:

- a struct of type `chirp_t` defined in `chirp.h`
- starting frequency
- end frequency
- signal length
- loop-time in microseconds

This initialises the values in the data struct and:

- converts loop period from microseconds to seconds
- calculates the number of samples in the signal duration
- calculates the exponential growth factor
- precomputes the constants used to get the phase from the frequency evolution
- calls `chirpReset()` function

The `chirpReset()` function sets:

- internal counter to 0
- bool `isFinished` to false

then calls the function `chirpResetSignals()` to set signal-related fields to 0

Once the chirp is initialised use the `chirpUpdate()` function once per loop iteration. This function checks:

- if bool `isFinished` is true -> return false
- else if internal counter is equal to the number of samples inside the chirps period
-> set bool `isFinished` true, call `chirpResetSignals()`, return false

if none of the above is true the ...

5 Theory

5.1 Signal Processing

Text....

5.1.1 Data Import

Text....

5.1.2 Handle High-Resolution Data

Text...

5.1.3 Time Conversion and Sampling Rate Verification

Text...

5.1.4 Application of Rotational Filtering (Apply Rotfiltfilt)

Text...

5.1.5 Estimation of Frequency Response

Text...

5.2 Control System

Text...

5.2.1 Transfer Function Estimation

Text...

5.2.2 Closed Loop Calculation

Text...

5.2.3 Filters and Controllers

Text...

5.2.4 Compensation of the I-Term

Text...

5.3 Evaluation

Text...

5.3.1 Step Response

Text...

5.3.2 Spectra Analysis

Text...

5.3.3 Spectrum Analysis

Text...

6 Challenges and Methodological Approach

6.1 Simulations

Text...

6.1.1 Simulation of Betaflight Controller Order

Text....

6.1.2 Simulation PID Controller

Text...

6.1.3 Simulation Frequency Response

Text....

6.1.4 Simulation Spectra

Text....

6.1.5 Simulation Spectrogram

Text....

6.2 Building an FPV drone

Text...

6.3 Tuning an FPV drone

Text...

6.4 Decode

Text...

6.5 Code Structure and programming style

Text...

6.5.1 OOP vs. Functional Approach

Text...

6.5.2 Final Structure Concept

Text...

6.6 Conversion MATLAB to Python

Text...

6.7 Python in MATLAB

Text...

6.8 Transfer MATLAB Library to Python

Text...

7 Results

Text...

8 Discussion and next steps

8.1 Discussion

Text...

8.2 Next Steps

Text...

8.3 Conclusion

Text...

9 Directories